

Forward Deployed Engineer - Take-Home Assignment

Role: Forward Deployed Engineer, AI Implementation Team

Platform: You may use any major AI platform (Claude, Gemini, OpenAI, etc.) and any agentic framework of your choice (e.g., LangChain, CrewAI, AutoGen, Claude Code, OpenAI Agents SDK, etc.)

Customer Onboarding Automation for a Mid-Market SaaS Company

The Scenario

You are an FDE embedded with **NovaCRM**, a mid-market CRM platform with ~200 customers. Their Customer Success team currently runs a manual onboarding process for every new customer. Here is what their CS Manager, Priya, told you during a discovery call:

"When a new deal closes, our AE sends an email to our CS team inbox with the customer name and the Salesforce opportunity link. From there, we manually create a project in Asana with our standard onboarding template - about 15 tasks across 4 phases: Kickoff, Data Migration, Configuration, and Go-Live. We then create a shared Slack channel with the customer and schedule the kickoff call. Each phase has its own checklist, and we update Asana as we go. If a task is blocked for more than 3 days, we're supposed to escalate - but honestly, that falls through the cracks a lot. At the end of onboarding, we write a handoff summary and move the customer to our ongoing support team."

She also mentioned:

"Some customers are on our Enterprise plan and get a dedicated CSM and a 30-day onboarding. Others are on Growth and get a pooled CSM with a 14-day onboarding. We sometimes mix up the templates. And we've had cases where data migration tasks were marked done but the customer's data wasn't actually verified - that caused a few ugly escalations."

Your Task

Part 1 - Requirements & Workflow Analysis (submit a document)

Based on Priya's description above, document your understanding of the workflow and your proposed solution. You may ask as many clarifying questions as you need to karthikr@rocketlane.com - include in your document whatever assumptions you made. Your analysis should cover your understanding of the current workflow, your proposed agentic workflow.

Part 2 - Build the Agent System

Build a working multi-agent system that automates this onboarding workflow. At minimum, your system should include:

- **Agent 1 - Intake & Routing Agent:** Monitors a Gmail inbox for new deal notification emails from AEs. When a new email arrives, the agent parses the customer name, customer contact email, AE name, and other details. However, the email does not include the plan tier. The agent must **place a phone call to the AE** using any Voice AI tool to verbally confirm whether the customer is on the Enterprise or Growth plan before proceeding. Once the plan tier is confirmed, the agent creates a project in **Rocketlane** via the Rocketlane API with the appropriate onboarding template and timeline.
- **Agent 2 - Communication Agent:** Creates a shared Slack channel with the customer for onboarding communication. The channel name, topic, and initial welcome message should be personalized based on the plan tier and customer details.
- **Rocketlane Automation - Overdue Task Escalation:** Configure an automation within Rocketlane (not a separate agent) that handles overdue task escalation: when a task becomes overdue by 1 day, notify the Project Manager; when a task becomes overdue by 4 days, notify the Project Owner.

Part 3 - Guardrails

Implement at least the following guardrails (and add any others you think are important):

- The Intake Agent must validate that all required fields are present before proceeding. If data is missing, it should ask for clarification rather than guessing.
- The Intake Agent must not proceed with project creation until it has received a clear, unambiguous plan tier confirmation from the AE via the voice call. If the call fails or the response is unclear, the agent should retry or escalate to a human rather than assuming a tier.
- All agent actions should be logged with timestamps, inputs, outputs, and decision rationale.

Part 4 - Automated Tests

Write automated tests that cover:

- **Happy path:** A new Enterprise deal email arrives, the Intake Agent parses it, the voice call confirms the plan tier, the project is created in Rocketlane with the right template, and the Communication Agent creates a properly personalized Slack channel.
- **Validation:** Intake Agent correctly rejects incomplete or malformed emails (missing customer name, no contact email, etc.).
- **Template accuracy:** When the AE confirms Enterprise, the agent creates the 30-day template with dedicated CSM; when the AE confirms Growth, it creates the 14-day template with pooled CSM.
- **Edge case:** What happens if the Rocketlane API is down? What if the customer already has an existing onboarding project? What if the AE doesn't answer the phone call, or gives an ambiguous response about the plan tier?

Deliverables

1. Written requirements analysis document (Part 1)
2. A video walkthrough demonstrating two scenarios end-to-end:
 - **Positive sample:** A complete happy-path run where everything works correctly - email arrives, voice call succeeds, Rocketlane project is created with the right template, Slack channel is set up, and the Rocketlane automation is shown.
 - **Negative sample:** A run where things go wrong - e.g., malformed email input, AE doesn't answer the call, ambiguous plan tier response, missing fields - showing how your system handles failures, retries, and guardrails.

Submission Guidelines

- Submit your video walkthrough via Loom or any screen recording tool. The video should clearly show both the positive and negative sample runs.
- For the voice call to the AE, you can use any Voice AI tool to build a real, working voice interaction - not a text simulation. Most Voice AI tools have a free tier
 - You can sign up for a free Rocketlane trial account for the project management integration. Use the Rocketlane API to create projects, tasks, and phases.
- For the Gmail trigger, you can use any Gmail account. The agent should monitor the inbox and automatically kick off the workflow when a deal notification email arrives.
- If you use mock/simulated APIs for secondary integrations (e.g., Slack), that's

acceptable - but explain in the video what would change in a production deployment.